Base de datos key-value replicada

PCP · Segundo cuatrimestre 2026 · Diseño según consigna V5

1. Estado, interfaz y TTL

La implementación corre en una única VM BEAM. Se asume que los procesos no fallan y que stop/1 se invoca una vez, al final, con todas las réplicas. simple_kv exporta start_link/2, stop/1, put/3, put/4, delete/2 y get/2.

start_link(Count, Alfa) valida Count entero positivo y Alfa entero ≥3 en el invocante, crea Count procesos con spawn_link y comunica sus pares y capacidad. Los TTL admitidos son enteros no negativos e infinity; cualquier otro falla con function_clause antes de enviar mensajes. put/3 usa infinity. Claves y valores admiten cualquier término Erlang.

Cada réplica mantiene exclusivamente en argumentos de su ciclo:

• Peers: identificadores de las otras réplicas; Alfa: capacidad máxima.

• Map: Key => {Version, {value, Val, Vence}}. Es el único almacenamiento de valores de la réplica; contiene como máximo Alfa entradas.

• Metadatos: Key => {Version, valor | borrado | desalojado}. Conserva versión incluso sin valor: un desalojo es local, un borrado invalida el dato.

• Pendientes: ConsultaRef => {Client, Ref, Key, Faltan}. Registra lecturas remotas; no almacena resultados ni valores.

Los metadatos pueden crecer con la cantidad de claves conocidas. No se recolectan lápidas: quitarlas sin demostrar que ya no quedan mensajes antiguos permitiría resurrecciones. La cota Alfa corresponde a valores, no a claves o versiones de metadatos.

El vencimiento se calcula como monotonic_time(millisecond) + TTL una única vez al escribir; infinity se conserva. Todas las réplicas comparten reloj monotónico en la VM. Replicar u ofrecer conserva el instante absoluto: recibir una copia no renueva su vida.

Un valor es vigente cuando ahora < Vence. La API vuelve a comprobar TTL al finalizar una lectura, incluso si el candidato llegó antes y venció mientras faltaban respuestas. TTL cero reemplaza lógicamente una escritura anterior por información vencida, sin desalojar otra clave. Los vencidos se retiran también al reservar espacio y recibir ofertas, conservando versión como borrado.

No se usan behaviours OTP, ETS, DETS, Mnesia, persistent_term ni diccionario del proceso. Los módulos auxiliares operan sobre los mapas recibidos como argumentos.

2. Protocolo de comunicación

Abreviaturas: C = cliente, R = réplica, P = par, I = invocante de inicio/cierre. E = {Version, {value, Val, Vence}} o {Version, borrado}. Resultado = error o {ok, Entry}, incluyendo marcas borrado/desalojado sin valor.

Mensaje	Emisor → receptor	Efecto
{peers, Peers, Alfa}	I → R	Inicializa pares y capacidad; empieza el ciclo.
{put, C, Ref, Key, Val, TTL}	C → R	Genera versión, reserva espacio y guarda; confirma y replica.
{delete, C, Ref, Key}	C → R	Genera lápida, elimina valor, confirma y replica.
{Ref, ok}	R → C	Termina put/delete sin esperar confirmaciones de pares.
{get, C, Ref, Key}	C → R	Responde con lo local o inicia consulta a pares.
{Ref, candidato, Resultado}	R → C	Entrega un candidato de lectura; el cliente reduce por versión.
{Ref, fin}	R → C	No quedan respuestas: el cliente comprueba TTL y retorna.
{replica, Key, E}	R → todos sus P	Aplica versión aceptable; controla capacidad si es valor. No confirma ni vuelve a replicar.
{ofrecer, Key, E}	R → todos sus P	Ofrece un valor antes de descartarlo. Se acepta solo vigente, sin versión más nueva y con espacio o reemplazo. No se reenvía.
{consultar, R, ConsultaRef, Key}	R → P	P contesta solo con su estado local; no inicia otra consulta.
{respuesta_consulta, ConsultaRef, P, Resultado}	P → R	Si P falta en esa consulta, reenvía candidato a C y elimina P de Faltan.
stop	I → R	R termina normalmente. No manda confirmación explícita.
{'DOWN', Monitor, process, R, Motivo}	VM → I	Confirma terminación de R al monitor creado por stop/1.

Cada operación usa make_ref(). Las consultas remotas tienen una referencia distinta de la del cliente. Una respuesta solo se acepta si la consulta existe y su emisor está en Faltan; así se descartan referencias desconocidas, duplicados y pares ajenos. Al terminar se elimina Pendientes[ConsultaRef].

El ciclo descarta mensajes inesperados; la inicialización también descarta mensajes que no sean su configuración. No hay reintentos ni confirmaciones para replicaciones/ofertas. La comunicación asincrónica se implementa con !; las funciones públicas esperan su propia referencia. stop/1 monitorea, envía stop y espera DOWN sin imponer un timeout que contradiga la espera de terminación.

3. Versiones, conflictos y borrados

Cada put/delete crea una versión con erlang:unique_integer([monotonic]). En la VM es única y creciente. La réplica de origen la genera una sola vez y todos los mensajes conservan esa versión. No se usa el orden de llegada como criterio de conflicto.

Una versión mayor reemplaza la información anterior; una menor se ignora. Para versiones diferentes, elegir el máximo es conmutativo, asociativo e idempotente: reordenar o duplicar mensajes no cambia el ganador. No hay empate entre dos escrituras originales distintas. Las versiones iguales provienen de copias de una misma operación.

Un delete produce {Version, borrado}. Aunque se elimine el valor de Map, Metadatos conserva la lápida. Un put atrasado con versión menor pierde contra esa lápida. Una nueva escritura posterior sí gana, como exige la consigna. Recibir un delete viejo tampoco elimina un valor nuevo.

Desalojar conserva {Version, desalojado}. Esta marca no significa que el dato dejó de existir: habilita la consulta a pares y la recuperación de una copia de igual versión. Para igual versión, aplicar_actualizacion/4 solo repone un valor cuando el estado era desalojado. elegir_mayor/2 prefiere el valor a la marca de desalojo de esa misma versión. Una lápida de borrado no se revierte mediante esa excepción.

La limpieza de un valor vencido conserva su versión como borrado. No crea una escritura nueva. Una copia de igual versión tiene el mismo vencimiento absoluto y tampoco es visible después de ese instante, aun si aún conserva físicamente el valor.

Convergencia y capacidad: alcance de la garantía

El criterio exigido es igualdad de los mapas de valores al cesar las escrituras y procesarse los mensajes. Acuerdo en versiones con distintas copias físicas no se considera suficiente. La política selecciona las Alfa versiones mayores, sin proteger por origen local.

Para puts de claves distintas con TTL infinito, cada recepción retiene el máximo conjunto de hasta Alfa entradas por versión entre las ya vistas. Incorporar una nueva entrada y quitar el mínimo equivale a calcular top-Alfa. Esta operación es independiente del orden y de duplicados; después de recibir el mismo conjunto, los mapas son iguales. Una versión menor para una clave ya actualizada no reemplaza la mayor.

No se extiende esta demostración automáticamente a borrados, sobrescrituras ni vencimientos: descartar una copia y luego eliminar otra clave puede dejar una vacante sin recuperar la anterior. Caso comprobado con Alfa=3: a1,b2,c3,d4,delete(d)5 produce b,c; delete(d)5,a1,b2,c3,d4 produce a,b,c. La versión d4 se ignora en el segundo orden por la lápida d5. Todos recibieron las mismas operaciones, pero los mapas difieren.

Falta resolver recuperación/reconciliación de copias y limpieza de TTL para cerrar igualdad física final. La V5 permite pérdida por saturación, pero eso no demuestra igualdad de los conjuntos restantes. El documento registra este límite; los tests aprobados no certifican convergencia integral.

4. Capacidad y selección por versión

La réplica reserva lugar antes de insertar. Primero retira vencidos. Si la clave ya está o queda una vacante, aplica la escritura aceptable. Si está llena, compara la versión entrante con la menor versión de todo el mapa. Si es mayor, ofrece y desaloja esa entrada antes de guardar; si es menor, ofrece y descarta el propio valor recibido, conservando versión y marca de desalojo.

La regla no distingue put local de replicación remota. No existe estado Locales ni cupo local protegido. Un put local genera la versión más alta de la VM y entra; una replicación atrasada puede perder frente a todas las residentes. La selección no depende del orden de las claves ni del TTL relativo: el TTL solo determina vigencia.

Cota de almacenamiento

El estado inicial tiene cero valores. Cada inserción reemplaza una clave existente, usa una vacante o retira una entrada antes de agregar otra. Nunca se construye un mapa con Alfa+1 entradas. Deletes y limpieza reducen tamaño; una oferta solo entra con espacio o reemplazo. Por inducción, todos los estados de almacenamiento respetan Alfa.

La selección mantiene las Alfa versiones mayores entre los candidatos recibidos y disponibles en el escenario de inserciones sin invalidaciones. No puede reconstruir una copia descartada cuando posteriormente queda espacio: ese es el pendiente de reconciliación expuesto en la sección 3.

Ofertas y terminación

Antes de descartar un valor vigente se lo ofrece a todos los pares. Una oferta se acepta si está vigente, no hay una versión posterior y hay espacio o reemplaza una clave existente. Procesarla no provoca desalojos ni reenvíos; por eso no desencadena una cadena de mensajes. Las replicaciones no se vuelven a replicar. Para un número finito de escrituras hay un número finito de ofertas.

Una oferta no funciona como reconciliación completa: un nodo lleno puede rechazarla y quedar con espacio después. No se afirma que las ofertas por sí solas cierren el pendiente de convergencia.

Requisito del 75 % local

Se adopta selección global por mayor versión para buscar mapas iguales. La redacción del 75 % local requiere aclaración frente a ese objetivo: con Alfa=3, dos nodos que escribieron tres claves distintas cada uno no pueden conservar sus seis claves en un mapa común de tres entradas. No se garantiza protección independiente de las últimas escrituras locales frente a todas las remotas. Esa interpretación queda explícitamente pendiente, en vez de introducir una excepción que rompe la igualdad final.

5. Lecturas, progreso y validación

Si el estado local es valor o borrado, la réplica envía candidato y fin al cliente. Sin pares también responde con lo local. Si es desconocido o desalojado, envía el candidato local inicial, registra Pendientes y consulta una sola vez a cada par.

Cada par contesta inmediatamente con lo que conoce, sin consultar a otros. La réplica solicitante reenvía cada respuesta válida al cliente y actualiza Faltan. Al llegar la última, vuelve a leer su estado local (pudo cambiar mientras esperaba), envía ese candidato y después fin. Todos esos mensajes hacia el cliente salen del mismo proceso; el orden de Erlang garantiza que fin no adelante candidatos anteriores.

La reducción por versión ocurre en la función pública get/2, en el proceso cliente, no en la réplica. El cliente mantiene el mejor resultado hasta fin y comprueba su vencimiento justo antes de retornar {ok, Val} o not_found. Pendientes solo conserva claves, referencias e identificadores: los resultados remotos no constituyen una caché ni almacenamiento secundario de valores del nodo. Los mensajes en tránsito son parte del protocolo; no se guardan en otra estructura de la réplica.

No hay lectura con garantía de instantánea o linealizabilidad entre escrituras concurrentes; se usa la mejor información recibida y una comprobación local final. Cuando cesan las escrituras y llega la propagación, la igualdad final de mapas solo se considera justificada en el alcance indicado en la sección 3.

Ausencia de deadlock

La réplica que consulta vuelve al ciclo normal: puede atender puts, deletes, ofertas y consultas mientras faltan pares. Responder una consulta no depende de una respuesta de otro proceso. Por eso no hay arista de espera bloqueante entre réplicas, aun cuando se consulten mutuamente. Con procesos que no fallan y ejecución eventual de los mensajes, todos los pares responden; el cliente recibe fin. No se promete latencia acotada ni recuperación de procesos fallidos, que quedan fuera de los supuestos.

Validación reproducible

rebar3 compile y rebar3 eunit compilan con warn_all y warnings_as_errors. También se puede compilar directamente con erlc +warn_all -Werror. Los tests cubren API y validaciones, enlaces, términos compuestos, TTL y reemplazos, propagación, conflictos desordenados/concurrentes, borrados sin resurrección, capacidad receptora, selector, ofertas y lecturas remotas.

Las regresiones V5 cubren el límite y las mayores versiones para Alfa de 3 a 16, órdenes opuestos, tráfico remoto y mapas locales iguales con escritores concurrentes, TTL cero con capacidad llena, cierre demorado más de un segundo, descarte durante inicialización, respuestas duplicadas/ajenas y vencimiento de un candidato mientras falta otro par. Se combinan pruebas por API pública con pruebas internas del protocolo para forzar órdenes concretos; las internas complementan, no sustituyen, el comportamiento observable.

Las pruebas finitas no demuestran todos los intercalados. La cota, elección de versiones y terminación se justifican por las invariantes anteriores; el cierre integral de convergencia sigue pendiente. La verificación administrativa y el push al repositorio asignado por la cátedra corresponden a los integrantes; el repositorio privado de trabajo no constituye por sí solo la entrega.